# Breadth-First Search

## Graph Representations

- Before we can run algorithms like BFS or DFS, we must decide how to store the graph's structure in a computer's memory

### 1. Adjacency List:

- For each vertex, store a list of its neighbouring vertices
- **Formal Definition:** A collection of $|V|$ linked lists, one for each vertex $v \in V$, where each list contains all vertices $u$ such that there exists an edge $(v, u) \in E$

### 2. Adjacency Matrix:

- **Formal Definition:** A square matrix of $A$ of size $|V| \times |V|$ such that $A[i, j] = 1$ if there is an edge between vertex $v_i$ and $v_j$, and $A[i, j] = 0$ otherwise
- **Note:** Directed edges start from the row and end at the column

### 3. Edge Array (Edge List):

- The set of edges $E$, where each element is a pair of connected vertices $(u, v)$

---

### Example: Graph Representation

- Consider the directed graph $G$ (with a self-loop on node `2`):

```
flowchart LR
    %% Node Definitions
    1((1))
    2((2))
    3((3))
    4((4))
    5((5))

    %% Top Row
    1 --> 2

    %% Vertical/Diagonal Links
    1 --> 3
    2 --> 4

    %% Bottom Row
    3 <--> 4
```

```
    4 --> 5

    %% Self loop
    2 --> 2
```

**Adjacency List Representation:**

```
1: [2, 3]
2: [2, 4]
3: [4]
4: [3, 5]
5: []
```

**Adjacency Matrix Representation:**

For 5 vertices, we use a $5 \times 5$ grid:

$$\begin{array}{cc} & \text{End (j)} \\ & \begin{array}{ccccc} \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{4} & \mathbf{5} \end{array} \\ \text{Start (i)} \begin{array}{c} \mathbf{1} \\ \mathbf{2} \\ \mathbf{3} \\ \mathbf{4} \\ \mathbf{5} \end{array} & \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

**Edge Array (Edge List) Representation:**

$$E = \{(1,2),(1,3),(2,2),(2,4),(3,4),(4,3),(4,5)\}$$

# Breadth-First Search (BFS)

- An algorithm for traversing or searching graph data structures
- BFS starts at a source node and explores all neighbouring nodes at the current depth before to moving on to the nodes at the next depth level

## Vertex Colouring System:

To keep track of progress and avoid infinite loops, BFS uses a three-colour system for every vertex:

- **White (Undiscovered):** The vertex hasn't been seen by the search yet
- **Grey (Discovered / Frontier):** The vertex has been found and added to the queue, but its neighbours haven't been fully explored yet
- **Black (Explored / Finished):** The vertex and all of its immediate neighbours have been fully processed

**Frontier Vertices (Grey):**

- The set of unvisited vertices that are adjacent to the vertices currently being explored

### Data Structures Used:

- **Queue** $(Q)$**:** A First-In-First-Out (FIFO) structure used to manage the "Grey" frontier of vertices waiting to be processed
- **Distance Array** $(d)$**:** Stores the shortest distance (number of edges) from the source to each vertex
- **Predecessor Array** $(\pi)$**:** Stores the "parent" of each node (the node that first discovered it) to allow for path reconstruction back to the source

---

## Algorithm: Breadth-First Search (BFS)

- BFS systematically builds a ***Breath-First Tree*** where the path from the source to any node is guaranteed to be the shortest possible path (number of edges in the path)

### Method:

1. **Set up:** Set all vertices to **White** (undiscovered), and all distances $d$ to $\infty$, and initialise the set of predecessors $\pi$ to be $NIL$ (empty set)
2. **Source Vertex:** Set the source vertex $s$ to be **Grey** (discovered), its distance $d[s] = 0$, and enqueue it into $Q$
3. **Discovery Loop:** While the queue $Q$ isn't empty, dequeue the head vertex $u$
4. **Current Neighbours:** For each neighbour $v$ in the adjacency list of $u$:
   - If $v$ is **White**, (newly discovered):
     - Set its colour to **Grey**
     - Calculate its distance $d[v] = d[u] + 1$
     - Set its predecessor $\pi[v] = u$
     - And enqueue it
5. **Completion:** Once all neighbours of $u$ are checked, colour $u$ **Black** (finished) and repeat $\mathbf{step\ 3}$ until the queue it empty

**Queue** $(Q)$**:**

- The Queue $Q$ stores the discovered vertices (that haven't yet been chosen) in the order that they are going to be chosen
- **FIFO** Queue so the oldest, and therefore closest, vertices are chosen first

---

### Code Implementation: BFS (Tested in VS Code)

```python
# import the Double-Ended Queue (for a real implementation)
from collections import deque

# Define the basic Graph class
class Graph:
    def __init__(self, vertices, adjacency_list):
        self.vertices = vertices # List of vertices
        self.adjacency_list = adjacency_list # Dictionary

# Let G be the Graph, and s be the source node
def breadth_first_search(G, s):

    # Step 1: Set up with Empty Dictionaries
    colour = {}
    d = {}
    pi = {}

    # Initialise all vertices to White, distance infinity, predecessor None
    for v in G.vertices:
        colour[v] = "WHITE"
        d[v] = float('inf')
        pi[v] = None

    # Source Vertex: Set s to Grey, distance to 0
    colour[s] = "GREY"
    d[s] = 0
    pi[s] = None

    # Setup Queue: We use a deque (double-ended queue) instead of a standard
Python list
    # because removing from the front of a list is O(n), whereas
deque.popleft() is O(1)
    Q = deque()
    Q.append(s)

    # Discovery Loop: While Q isn't empty, dequeue the head vertex u
    while len(Q) > 0:
        u = Q.popleft()

        # Current Neighbours: For each neighbour v of u
        for v in G.adjacency_list[u]:
            # If v is White (newly discovered), update and enqueue
            if colour[v] == "WHITE":
                colour[v] = "GREY"
                d[v] = d[u] + 1
                pi[v] = u   # Set u to be its predecessor
                Q.append(v)

        # Completion: Colour u Black (finished)
```

```
        colour[u] = "BLACK"

    return d, pi
```

### Run Code

```
# ----------------------------------------------------------
# Works with the code above (copy-paste into VS code)
# ----------------------------------------------------------
# Initialise G using the graph from earlier
example_vertices = [1, 2, 3, 4, 5]
example_adj_list = {
    1: [2, 3],
    2: [2, 4], # Includes the self-loop
    3: [4],
    4: [3, 5], # 3 and 4 share a bidirectional edge
    5: [] # Sink node
}
G = Graph(example_vertices, example_adj_list)

# Run the search starting from vertex 1
distances, predecessors = breadth_first_search(G, 1)

print("Shortest Distances from Source (1):")
print(distances)

print("\nPredecessor (Parent) Nodes:")
print(predecessors)
```

---

## BFS Time & Space Complexities

### Time Complexity: $O(V + E)$

- **Vertices:** Each vertex enters and leaves the queue exactly once, which takes $O(V)$ time
- **Edges:** The adjacency list of each vertex is scanned exactly once during the entire process, meaning all edges are processed in $O(V)$ time

### Space Complexity: $O(V)$

- The **Auxiliary Data Structures:** the Queue ($Q$), distance array ($d$), and predecessor array ($\pi$) all scale linearly with the number of vertices

---

## Auxiliary Space Complexity:

- **Total Space Complexity:** The memory taken up by *both* the input data, ***and*** the extra memory used by the algorithm
- **Auxiliary Space Complexity:** *Only* the extra memory used by the algorithm while it runs
- **Auxiliary Data Structures:** "Helper" data structures introduced by an algorithm to help with execution and solve a specific problem

## Examples of Auxiliary Data Structures: Graph Traversals

When a BFS or DFS algorithm runs, they can't modify the input Graph $G$, so they use auxiliary data structures to keep track of things:

- **Queue** $(Q)$ **or Stack:** Used to manage the order in-which vertices are being processed
- **Colour Dictionaries:** Used to track if a vertex is White, Grey, or Black
- **Distance or Time Arrays** $(d, f)$**:** Used to record edge distances or timestamps
- **Predecessor Array** $(\pi)$**:** Used to remember the "parent" nodes to help reconstruct the final path

---

**Related Pages:**

- Intro to Graph Theory
- Shortest Paths, Cycles & Connectivity
- Trees & Isomorphism
- Depth-First Search
- Minimum Spanning Trees